

- **A comprehensive toolkit for streamlined development:** Bun is not merely a runtime; it's a comprehensive toolkit that covers all aspects of the development process. It offers an all-in-one approach, encompassing a package manager, bundler, and test runner. This integrated solution simplifies and streamlines development, eliminating the need to juggle multiple tools and processes. Bun's developers have recognised that an efficient workflow is crucial for modern web and application development.
- **Native TypeScript support:** One distinctive feature that sets Bun apart from Node.js is its native support for TypeScript (TS). Unlike Node.js, which may require third-party transpilers or complex configurations, Bun embraces TypeScript natively. This means that developers working with TypeScript can do so seamlessly and efficiently, without additional layers of complexity.
- **Expanding possibilities with .jsx and .tsx files:** Another remarkable aspect of Bun is its support for .jsx and .tsx files. This feature enables Bun to natively convert HTML and markup into JavaScript, which is a functionality not readily available in Node.js.
- **Efficiency meets memory management:** While Node.js is known for its minimal memory footprint, Bun has taken memory efficiency a step further. It's designed to be even more memory-efficient, addressing the needs of developers who demand optimal resource utilisation.
- **Interoperability and compatibility:** Bun aims for seamless compatibility with Node.js, making the transition from one to the other as smooth as possible. This allows developers to experiment with Bun without disrupting their existing workflows or investments in Node.js-based projects.

## 'Baking' Bun for your JS project

Getting started with Bun is as simple as running the 'bun init' command. This command scaffolds an empty project, providing developers with a ready-to-go environment. However, before that, we need to install Bun.

To install Bun on Linux/macOS/WSL you can use curl:

```
curl -fsSL https://bun.sh/install | bash
```

Bun also has a Docker image; to use it, simply run:

```
docker pull oven/bun
docker run --rm --init --ulimit
memlock=-1:-1 oven/bun
```

Interestingly, you can even install it using its competitor npm:

```
npm install -g bun
```

If you're a Windows user, note that at the time of writing this article, Bun only has an experimental build for Windows, which contains the Bun

runtime only. However, this is expected to change in the near future.

You can check the installation using the command:

```
bun --version
```

## Writing a simple HTTP web server in Bun

Let's understand Bun by writing a simple HTTP web server in it. To create a Bun project, we use the *init* command as follows. You can also refer to Figure 1 for clarity.

```
mkdir OSFY-bun
cd OSFY-bun
bun init
```

Here, I created a directory named 'OSFY-bun' and executed the *init* command there. This is essentially a setup command that asks for multiple questions related to our project. When naming the entry point (which is the main file we are going to execute), Bun accepts both .ts and .js for TypeScript and JavaScript, respectively. In the

```
sreyas@de4bian: ~/Desktop/OSFY-bun
sreyas@de4bian:~/Desktop/OSFY-bun$ bun init
bun init helps you get started with a minimal project and tries to guess sensible defaults. Press
^C anytime to quit

package name (osfy-bun): osfy-bun
entry point (index.ts): index.js

Done! A package.json file was saved in the current directory.
+ index.js
+ .gitignore
+ jsconfig.json (for editor auto-complete)
+ README.md

To get started, run:
  bun run index.js
sreyas@de4bian:~/Desktop/OSFY-bun$ bun run index.js
Hello via Bun!
sreyas@de4bian:~/Desktop/OSFY-bun$
```

Figure 1: Creating a Bun project

```
index.js - OSFY-bun - Visual Studio Code
File Edit Selection View Go Run Terminal Help
EXPLORER
OSFY-BUN
  > node_modules
  > .gitignore
  > bun.lockb
  > index.js
  > jsconfig.json
  > package.json
  > README.md
index.js
1 console.log('Hello via Bun!');
2
```

Figure 2: Simple 'hello world' program